



Exercices de révision

Programmation Java

Interfaces, généricité, collections et API Stream

Public : L2 Informatique

Prérequis : classes et objets, héritage, collections de base

Durée conseillée : 3 h de travail personnel

Année universitaire : 2025-2026

RABOANARY Heriniaina Andry

Directeur de la Pédagogie et de l'Assurance Qualité

Consignes

Cette feuille couvre les quatre thèmes centraux du chapitre « Des interfaces aux Streams API » : le vocabulaire de l'abstraction (généricité, interfaces, programmation fonctionnelle), l'ordonnement d'objets avec `Comparable` et `PriorityQueue`, l'indexation par clé avec les implémentations de `Map`, et enfin le traitement déclaratif de données avec l'API `Stream`.

Le code Java attendu doit être *compilable* : déclarations d'attributs, types génériques et signatures complètes. Une réponse partielle correctement justifiée vaut mieux qu'un code complet mais faux — commentez vos choix de conception.

Comment travailler cette feuille

Ce sont des exercices de révision : aucun n'est noté. Traitez-les dans l'ordre — l'exercice 4 réutilise la classe écrite à l'exercice 2 et la structure conçue à l'exercice 3. Cherchez chaque question avant de consulter le corrigé ; une réponse fausse que l'on comprend vaut mieux qu'une réponse juste recopiée.

Exercice 1 — Compréhension

1) Donnez une définition claire et un exemple d'utilisation en Java pour chacune des notions suivantes :

- a. classe générique
- b. méthode abstraite
- c. interface
- d. interface fonctionnelle
- e. classe anonyme
- f. expression lambda
- g. programmation fonctionnelle
- h. fonction d'ordre supérieur

2) Analyses comparatives.

- a. Voici deux écritures d'une même classe « boîte ». Chacune contient une erreur sur la dernière ligne. Dites, pour chaque version, à *quel moment* l'erreur est détectée et par *qui* — puis concluez sur ce qu'apporte la généricité.

```
1 // ----- version A : sans generique -----
2 class Boite {
3     private Object contenu;
4     public void ranger(Object o) { this.contenu = o; }
5     public Object sortir()      { return contenu; }
6 }
7
8 Boite b = new Boite();
9 b.ranger("bonjour");
10 Integer n = (Integer) b.sortir();
11
12 // ----- version B : avec generique -----
13 class Boite<T> {
14     private T contenu;
15     public void ranger(T o) { this.contenu = o; }
16     public T sortir()      { return contenu; }
17 }
18
19 Boite<String> b = new Boite<>();
```

```

20 b.ranger("bonjour");
21 Integer n = b.sortir();

```

- Une expression lambda face à une classe anonyme implémentant la même interface fonctionnelle. Que gagne-t-on, et qu'est-ce qu'on ne peut plus faire avec la lambda ?
- Un tri par `Comparator` face à un tri par `Comparable`. Pourquoi le premier permet-il de trier une classe dont on ne possède pas le code source ?
- Les deux fragments ci-dessous produisent exactement le même résultat (vérifié). Comparez-les : que rend visible chacune des deux écritures, qu'est-ce qui disparaît en passant de l'une à l'autre ? Indiquez ensuite un type de traitement pour lequel la boucle resterait préférable au pipeline.

```

1 // ----- version A : boucle -----
2 Map<String, List<Smartphone>> parMarque = new HashMap<>();
3 for (Smartphone p : catalogue) {
4     if (p.getPrix() < 600) {
5         parMarque.computeIfAbsent(p.getMarque(), m -> new ArrayList<>()).add(p);
6     }
7 }
8
9 // ----- version B : pipeline -----
10 Map<String, List<Smartphone>> parMarque =
11     catalogue.stream()
12         .filter(p -> p.getPrix() < 600)
13         .collect(Collectors.groupingBy(Smartphone::getMarque));

```

Exercice 2 — Application de la classe `File` de priorité

1) Écrire une classe `Smartphone` (implements `Comparable<Smartphone>`) :

- Avec, au moins, les attributs suivants :
 - Marque (`String`)
 - Capacité batterie (en mAh)
 - Taille écran (en pouces)
 - Prix (en euros)
 - Année de sortie
- Avec deux méthodes de votre choix.
- Avec une redéfinition pertinente de la méthode `public String toString()`.
- Redéfinir la méthode `public int compareTo(Smartphone autre)`. On veut utiliser la classe `PriorityQueue<Smartphone>`. Redéfinir la méthode `public int compareTo(Smartphone autre)` héritée de l'interface `Comparable<Smartphone>`. On veut donner la priorité aux smartphones les plus récents.

Pour indication :

```

1 // on compare this a autre
2 // on retourne une valeur <0 si this est prioritaire par rapport a autre
3 // on retourne une valeur >0 si autre est prioritaire par rapport a this
4 // on retourne la valeur 0 sinon
5 @Override
6 public int compareTo(Smartphone autre) {
7     return ...
8 }

```

- 2) Si on voulait changer le critère de priorité, comment réécrire `compareTo()` pour chacune des règles suivantes :
- Ordre alphabétique des marques.
 - Priorité aux smartphones ayant la plus grande autonomie (capacité batterie). Si deux smartphones ont la même capacité, donner la priorité au smartphone ayant le plus grand écran. Et si les smartphones ont la même capacité batterie et la même taille d'écran, donner la priorité au smartphone le moins cher.

Exercice 3 — Indexation par une chaîne de caractères

Partie A — Retrouver un titre associé à un code

On veut écrire une classe `Bibliotheque` qui contient les paires (Code, Titre) pour chaque livre. On veut pouvoir retrouver rapidement le titre d'un livre à partir de son code.

- Créer un tableau comparatif entre `TreeMap` et `HashMap`.
- Expliquer en une phrase pourquoi un `HashMap<String, String>` serait plus performant qu'un `ArrayList<>` ou un `LinkedList<>`.
- Écrire une méthode qui permet d'ajouter une paire (Code, Titre) dans la bibliothèque.
- Écrire une méthode qui permet de retrouver rapidement le titre d'un livre à partir de son code.

Critères de comparaison suggérés pour la question 1

Structure interne, complexité de `get` et `put`, ordre d'itération, contraintes sur le type des clés, gestion de la valeur `null`, cas d'usage typique.

Partie B — Ajout d'une information supplémentaire et une nouvelle recherche

Dans la même classe, on veut également stocker l'auteur de chaque livre. De plus, on veut pouvoir retrouver rapidement tous les livres d'un auteur donné.

- Puisqu'on ne peut pas écrire `HashMap<String, String, String>`, proposer une solution. Ne pas implémenter mais bien expliquer votre conception (utiliser des diagrammes UML si nécessaire).
- Réécrire la classe `Bibliotheque` avec tous les attributs nécessaires dans le but d'ajouter une méthode qui permet de retrouver tous les titres (ou codes) des livres écrits par un auteur donné.

Point de vigilance

À la question 6, l'ajout d'un livre doit laisser la structure *cohérente* : toute information ajoutée d'un côté doit l'être de l'autre. Précisez ce que fait votre méthode d'ajout lorsqu'un auteur apparaît pour la première fois.

Exercice 4 — Les essentiels de l'API Stream

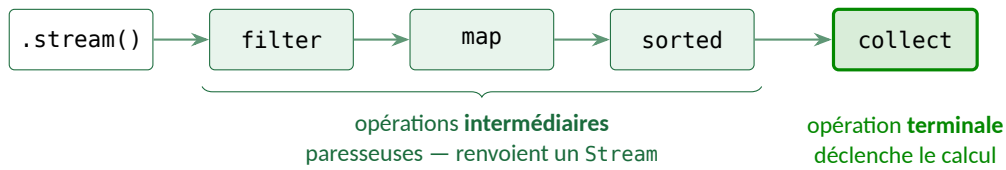
On reprend la classe `Smartphone` de l'exercice 2, munie des accesseurs `getMarque()`, `getBatterie()`, `getEcran()`, `getPrix()` et `getAnnee()`. On dispose d'un catalogue :

```
1 List<Smartphone> catalogue = List.of(
2     new Smartphone("Nokia", 5000, 6.5, 249.0, 2023),
3     new Smartphone("Xiaomi", 5000, 6.7, 399.0, 2024),
```

```

4 new Smartphone("Samsung", 4500, 6.1, 899.0, 2025),
5 new Smartphone("Nokia", 4000, 6.5, 179.0, 2022),
6 new Smartphone("Apple", 3300, 6.1, 1099.0, 2025));

```



Rien ne s'exécute tant que l'opération terminale n'est pas appelée.

Fig. 1 : Anatomie d'un pipeline Stream : une source, une suite d'opérations intermédiaires, une opération terminale unique. Schéma de principe.

Partie A — Compréhension

- 1) Qu'est-ce qui distingue une opération *intermédiaire* d'une opération *terminale* ? Donnez trois exemples de chaque, avec leur type de retour.
- 2) Le fragment suivant produit l'affichage reproduit juste après. Expliquez cet ordre : pourquoi « Avant collect » sort-il en premier, et pourquoi « Beta » est-il absent du résultat final ?

```

1 List<String> noms = List.of("Alpha", "Beta", "Gamma");
2 Stream<String> s = noms.stream().filter(n -> {
3     System.out.println("test " + n);
4     return n.length() > 4;
5 });
6 System.out.println("Avant collect");
7 List<String> resultat = s.collect(Collectors.toList());
8 System.out.println(resultat);

```

Affichage obtenu :

```

Avant collect
test Alpha
test Beta
test Gamma
[Alpha, Gamma]

```

- 3) Un même Stream peut-il être parcouru deux fois ? Autrement dit, que se passe-t-il si l'on applique une seconde opération terminale au flux déjà stocké dans la variable `s` ci-dessus ? Que faut-il faire lorsqu'on a besoin du même résultat à plusieurs reprises ?
- 4) Pourquoi `findFirst()` renvoie-t-il un `Optional<Smartphone>` plutôt qu'un `Smartphone` ? Citez deux façons correctes d'exploiter cet `Optional`, et une façon dangereuse.
- 5) Quel est le lien entre une interface fonctionnelle (exercice 1, question 1.d) et l'argument que l'on passe à `filter` ? Donnez le nom de l'interface concernée et sa méthode abstraite.

Partie B — Écrire des pipelines

Pour chaque question, écrivez une *seule* instruction utilisant l'API Stream, et précisez le type exact du résultat. Utilisez une référence de méthode (`Smartphone : :getPrix`) chaque fois que c'est possible.

- 6) La liste des smartphones à moins de 600 euros, triés du plus récent au plus ancien.
- 7) La liste des marques distinctes, par ordre alphabétique.

- 8) Le prix moyen du catalogue; puis le smartphone le plus cher.
- 9) Le nombre de smartphones sortis en 2025 ou après.
- 10) Une chaîne unique listant les marques distinctes séparées par une virgule et une espace — par exemple "Apple, Nokia, Samsung, Xiaomi".
- 11) La capacité batterie cumulée du catalogue. Écrivez deux versions : l'une avec `reduce`, l'autre avec un flux spécialisé de primitives (`IntStream`, obtenu par `mapToInt`). Laquelle préférez-vous, et pourquoi ?
- 12) Les réponses aux trois questions : « existe-t-il un smartphone à plus de 1000 euros ? », « tous ont-ils au moins 3000 mAh ? », « aucun n'est-il antérieur à 2020 ? ».
- 13) Une `Map<String, List<Smartphone>>` regroupant les smartphones par marque; puis une `Map<String, Long>` donnant le nombre de modèles par marque; puis une `Map<String, Double>` donnant le prix moyen par marque.
- 14) Traduisez la boucle suivante en un pipeline Stream équivalent :

```

1 List<String> resultat = new ArrayList<>();
2 for (Smartphone p : catalogue) {
3     if (p.getAnnee() >= 2024) {
4         resultat.add(p.getMarque().toUpperCase());
5     }
6 }
7 Collections.sort(resultat);

```

Partie C — Retour sur l'exercice 3

- 15) On dispose maintenant d'une `List<Livres>` livres, où `Livres` expose `getCode()`, `getTitre()` et `getAuteur()`. Écrivez en une instruction la construction de la structure `Map<String, List<String>>` associant chaque auteur à la liste des titres de ses livres.
- 16) Comparez cette construction à celle de l'exercice 3, question 6. Laquelle utiliseriez-vous si la bibliothèque évolue en permanence (ajouts et suppressions de livres) ? Justifiez en termes de coût de reconstruction.

Aide-mémoire — à ne pas confondre

- `filter` garde des éléments, `map` les transforme : le premier ne change pas le type, le second si.
- `sorted()` sans argument exige que le type implémente `Comparable`; sinon il faut lui passer un `Comparator`.
- `count()`, `collect()`, `forEach()`, `reduce()`, `anyMatch()` *consomment* le flux : il devient inutilisable.
- `Collectors.toList()` renvoie une liste modifiable, `Stream.toList()` une liste *non* modifiable.
- Un flux d'objets (`Stream<Integer>`) n'a ni `sum()` ni `average()` : passer par `mapToInt` / `mapToDouble`.